WHITE PAPER

Deliverables Store & Memory Layer

Extending Live Ninja with file creation & delivery and a personal memory layer — with a comparative analysis of memory architectures

Version: 1.0 • **Date:** 2026-07-17
Author: Jeremy Proffitt
Product: Live Ninja — GPT-Realtime voice assistant platform
Scope: Two new platform capabilities + a recommended memory design
Status: Draft for decision

Executive Summary

This white paper proposes two additions to the Live Ninja platform and recommends a concrete design for each.

1. A Deliverables Store. Today Live Ninja can *talk*; this capability lets it *produce*. The assistant gains the ability to **create files** (documents, reports, exports, transcripts, generated artifacts), **package** several of them into a single ZIP, and **deliver** them to the user as first-class, separately-stored downloadables. Every artifact-producing interaction — from any surface — writes a durable record to a per-user **Deliverables Store** on Amazon S3, indexed in DynamoDB, and browsable/downloadable from **both the website and the Android app**. Deliverables are grade-able, shareable, and independent of the conversation that produced them.

2. A Memory Layer. Live Ninja should remember — not as an undifferentiated chat log, but as a **structured personal memory** organized around the things a life is made of: **people, places, and information**, plus **organizational** (projects, lists, documents) and **planning** (goals, tasks, schedules) capabilities. This paper defines a memory taxonomy and then does the part you asked for: it **compares five candidate architectures** — a local RAG on your own machine, Amazon S3 Vectors, a DynamoDB entity graph, OpenSearch Serverless, and pgvector — on cost, latency, privacy, operational burden, and scale. It recommends a **hybrid**: a DynamoDB **entity-and-relationship graph** for structured/organizational/planning memory, **S3 Vectors** for low-cost semantic recall, and an **optional local RAG sidecar** on your PC for private, high-volume retrieval — all exposed to GPT-Realtime as callable tools.

Both features fit the platform's existing grain: serverless, S3-first, DynamoDB-first, no expensive managed services, cost-guarded, and OIDC-deployed.

Part I — The Deliverables Store

1. The capability

Three verbs, exposed to the assistant as server-side tools and to the user as UI:

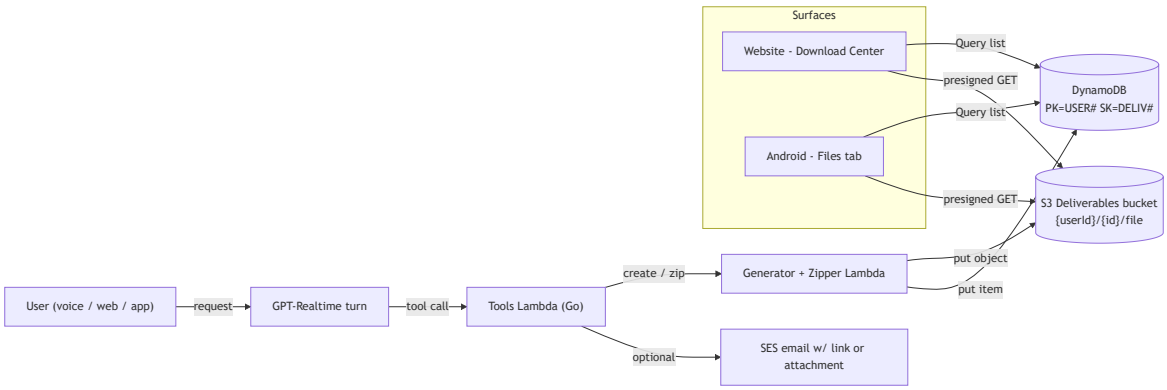
Verb	What it does	Tool (function-calling)
Create	Generate a file from a conversation or task — a PDF report, a Markdown doc, a CSV/JSON export, an ICS calendar, an image, a rendered artifact.	`deliverable.create(kind, name, content)`
Package	Bundle N previously-created files into one ZIP for a single clean download.	`deliverable.zip(ids[], name)`
Deliver	Persist the artifact, mint a time-boxed download URL, and surface it on every surface + optionally email it.	`deliverable.deliver(id, channels[])`

The assistant calls these during a turn ("Live Ninja, turn our chat into a one-page brief and zip it with last week's notes"), and the resulting files land in the store as **separate deliverables** — not buried in the transcript.

2. Architecture

The Deliverables Store reuses primitives already specified in the PRD (S3 uploads/downloads, DynamoDB single-table, presigned URLs, SES) rather than adding new infrastructure.

- **Storage — S3.** One bucket, owner-namespaced keys: `s3://live-ninja-deliverables/{userId}/{deliverableId}/{filename}`. Server-side encryption (SSE-S3/KMS), versioning, and a lifecycle policy for retention. ZIP packaging happens in a Lambda that streams member objects into an archive and writes the result back as its own deliverable.
- **Index — DynamoDB.** Each deliverable is an item under the user's partition so listing is a **Query, never a Scan**: `PK=USER#{userId}`, `SK=DELIV#{ts}#{deliverableId}`, with attributes `name, kind, sizeBytes, mimeType, sourceTurnId, sha256, status, expiresAt`. A GSI (`GSI1PK=DELIV#{deliverableId}`) supports direct lookup for share links.
- **Access — presigned URLs.** Downloads are short-lived presigned GETs authorized against the caller's `userId` prefix; uploads (when a client contributes a file) are presigned PUTs with a pinned content-type and size cap. No object is ever public.
- **"All transactions."** Every turn that produces an artifact writes a deliverable record; the store is therefore a complete, queryable ledger of everything Live Ninja has ever made for you — the same list rendered on web and app.
- **Delivery channels.** `web` (download center), `app` (Files tab), and `email` (SES attachment or link from a verified `@jeremy.ninja` identity).



diagram

3. UX

- **Website — Download Center:** a real sortable table (name, kind, size, created, source), row actions Download / Zip-selected / Share / Delete, empty/loading/error states, “Showing 1–25 of N”. Consistent with the platform’s rich-UI rules (no blind text boxes; structured presentation).
- **Android — Files tab:** the same list as cards, with a multi-select bar for “Zip & share.” Deliverables created by voice appear here automatically.
- **Cross-surface parity:** because the index is one DynamoDB partition, the identical list appears everywhere, instantly.

4. Security, retention, cost

Per-user isolation by key prefix and IAM condition; SSE + optional KMS; configurable retention via S3 lifecycle (e.g., 400-day default, “keep forever” flag per item); checksums (`sha256`) for integrity; virus/type allow-listing on any user-contributed upload. Cost is dominated by S3 storage (cents/GB-month) and negligible request costs — no always-on component. A CloudWatch size/emit alarm guards against runaway generation.

Part II — The Memory Layer

5. What "memory" means here

Not a chat log. Live Ninja's memory is a **personal knowledge base** whose first-class citizens mirror how you actually think:

- **People** — who they are, relationships, preferences, history ("my brother Jason", "Dr. Alvarez, cardiologist").
- **Places** — home, office, the cabin, frequent destinations, with attributes and geo.
- **Information** — facts, notes, documents, preferences, decisions ("I take the 8:10 train", "the router password is in the Deliverables Store").
- **Organizational capability** — projects, lists, documents, and the links among them.
- **Planning capability** — goals, tasks, reminders, schedules, and their status over time.

Underneath, four memory *types* serve these:

Type	Horizon	Example	Best store
Working	this session	current topic, last few turns	in-session context
Episodic	past events	"what did we decide Tuesday?"	transcripts + vectors
Semantic	durable facts	"Jason's birthday is March 3"	entity graph + vectors
Procedural	how-to / preferences	"always brief me in bullet points"	entity graph (prefs)

The design goal: at the start of every voice session, **retrieve the right slice of memory** (relevant people/places/facts/plans) and inject it into the GPT-Realtime persona; during the session, **write new memories** via tool calls; and let the user **see and edit** everything.

6. The options (the part you asked for)

Five candidate architectures for the *recall* substrate. Structured entity/relationship data (people, places, projects, tasks) always lives in **DynamoDB** — the debate below is really about **semantic vector recall** and where it runs.

Option A — Local RAG sidecar (on your PC)

A small service on your own machine: a local embedding model + a lightweight vector store (LanceDB, Chroma, `sqlite-vec`, or FAISS). The cloud backend calls home (or the machine pushes/pulls) for retrieval. - **Pros:** maximum privacy (raw notes never leave your hardware), near-zero marginal cost, fast local search, full control, works great for a single power user. - **Cons:** availability is tied to the PC being on and reachable; you must bridge it to cloud clients (a tunnel/queue, or treat it as authoritative and sync embeddings up); you own the ops. - **Fit:** excellent as an *optional* private, heavy-retrieval tier; risky as the *only* tier for an always-available assistant on a phone and an ambient device.

Option B — Amazon S3 Vectors (cloud-native, serverless)

AWS's native vector storage: "vector buckets" with built-in vector indexes and similarity search, priced to be dramatically cheaper than running a vector database, aimed at large or infrequently-queried vector sets. - **Pros:** serverless (nothing to run), S3-cheap storage, scales without capacity planning, fits the platform's S3-first ethos, integrates with Bedrock/OpenSearch if ever needed. - **Cons:** higher query latency than in-memory ANN (fine for assistant recall, not for tight loops); newer service — confirm GA/limits in `us-east-1`; not a graph. - **Fit:** the natural **default cloud recall tier** for this platform.

Option C — DynamoDB entity graph (no dedicated vectors)

Model people/places/facts as items and relationships as edges; retrieve by keys/GSIs and simple attribute filters; optionally store small embeddings as attributes and do brute-force cosine in the Lambda for tiny corpora. - **Pros:** already in the stack, cheapest for structured recall, perfect for organizational/planning data and exact lookups, no Scan needed with the right keys. - **Cons:** not a real ANN vector search; brute-force similarity doesn't scale past a few thousand vectors. - **Fit:** the **structured/organizational/planning backbone** — pair it with a vector tier, don't make it do semantic search alone.

Option D — OpenSearch Serverless (vector engine)

A managed vector + keyword hybrid search engine. - **Pros:** powerful hybrid (BM25 + vector) search, mature, filters + facets. - **Cons: cost** — a minimum OCU footprint means a meaningful monthly floor even when idle, which cuts against the "no expensive always-on infra" rule. - **Fit:** overkill and over-budget for a single-user personal assistant; revisit only at large scale.

Option E — pgvector on Aurora Serverless v2 / RDS

Postgres with the `pgvector` extension: relational data, vectors, and graph-ish queries in one engine. - **Pros:** one store for structured + semantic + relationships, SQL familiarity, strong ecosystem. - **Cons:** a database to run and pay for (Aurora Serverless v2 has a non-trivial idle floor; RDS is always-on); operational surface the rest of the platform avoids. - **Fit:** attractive if the platform ever wants one unified store, but it reintroduces an always-on cost the serverless design deliberately shed.

Comparison

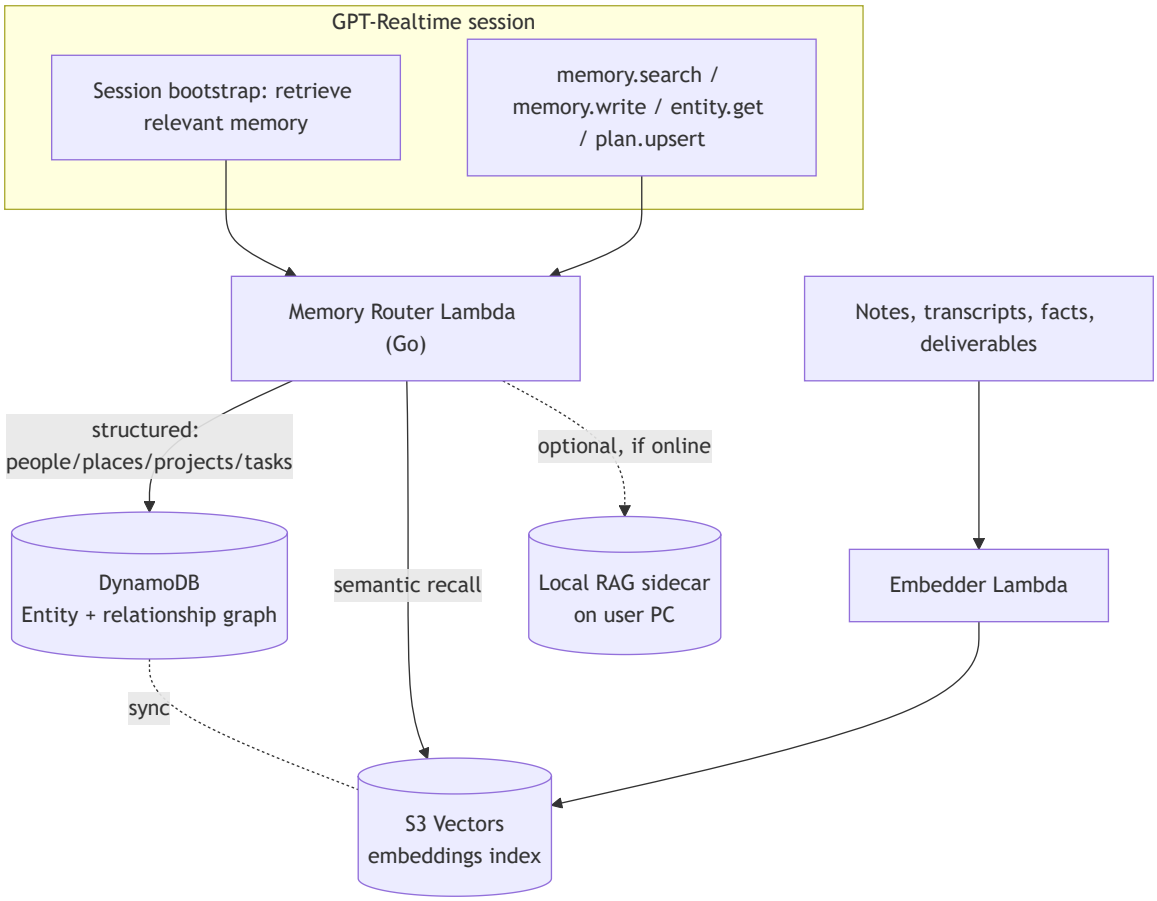
Option	Recall quality	Latency	Monthly cost (1 user)	Privacy	Ops burden	Scales
A · Local RAG	High	Low (local)	~$0	**Highest**	You run it	Per-machine
B · S3 Vectors	High	Medium	**Very low**	Cloud (encrypted)	None	**Yes**
C · DynamoDB only	Low (no ANN)	Low	**Lowest**	Cloud	None	Structured only
D · OpenSearch SL	**Highest**	Low	High (idle floor)	Cloud	Low	Yes
E · pgvector/Aurora	High	Low	Medium–High (idle floor)	Cloud	Medium	Yes

7. Recommendation — a hybrid, tiered memory

Use each store for what it is best at:

1. **DynamoDB entity graph** — the source of truth for **people, places, information, projects, tasks, plans** and the relationships among them. Structured, exact, cheap, already here. This delivers the *organizational* and *planning* capabilities directly (a project is an item; its tasks are child items; a person links to the projects they touch).
2. **S3 Vectors** — the default **semantic recall** tier. Every note, transcript chunk, and fact is embedded and indexed here for "what do I know about X?" retrieval, at S3 cost.
3. **Optional local RAG sidecar on your PC** — a private, high-volume tier you can switch on. When present and reachable it serves retrieval (and can hold material you never want in the cloud); when absent, the platform falls back to S3 Vectors seamlessly. This is the "RAG on my local computer" you flagged — supported as an *enhancement*, not a *dependency*.

Memory is exposed to GPT-Realtime as tools — `memory.search(query, entityTypes)`, `memory.write(fact, links)`, `entity.get(id)`, `plan.upsert(task)` — and a **session bootstrap** step retrieves the relevant slice (recent people/places/open tasks + top semantic hits) to prime the persona at connect time.



diagram

8. Privacy & control

Memory is personal and fully user-controlled: a **memory browser** on web/app lists every remembered person/place/fact/plan with edit and delete; a **redaction/forget** action removes an item from both DynamoDB and the vector index; the **local-RAG tier** lets designated material never touch the cloud; retention and export are one click (and an export is itself a Deliverable). On-device wake already guarantees only intended audio is ever processed; memory writes are always explicit tool calls, never silent capture.

Part III — How this folds into the plan

Both capabilities slot into the existing milestone structure without disturbing the core build:

Capability	PRD additions	Plan placement
Deliverables Store	`FR-DLV-*` (create/zip/deliver, store, list UI)	New **M9 — Deliverables Store** after core surfaces; reuses S3/DynamoDB/SES already in M0–M2
Memory Layer	`FR-MEM-*` (entity graph, semantic recall, memory tools, browser UI, privacy)	New **M10 — Memory Layer**; DynamoDB graph first, S3 Vectors second, optional local RAG third

Recommended sequence: ship the **Deliverables Store** first (small, high-value, reuses existing primitives), then the **Memory Layer** in three increments — entity graph → S3 Vectors recall → optional local RAG.

9. Recommendation summary

- **Build the Deliverables Store** on the S3 + DynamoDB primitives already specified; expose create/zip/deliver as tools and a Download Center / Files tab; make every artifact a separate, durable, cross-surface deliverable.
- **Build memory as a tiered hybrid:** DynamoDB entity/relationship graph (people, places, information, projects, tasks, plans) + S3 Vectors for semantic recall, with an **optional** local RAG sidecar on your PC for private/heavy retrieval.
- **Avoid** the always-on cost floors of OpenSearch Serverless and Aurora/pgvector unless scale later justifies them.
- Everything stays serverless, cost-guarded, S3/DynamoDB-first, and consistent with the platform's existing security, privacy, and deployment rules.